AMS 526: Numerical Linear Algebra for Computational and Data Sciences

Lecture 19: Computing the SVD; Sparse Storage Formats

Prof. Xiangmin Jiao

Stony Brook University

Outline

1. **Computing the SVD (NLA§31)**
2. Sparse Storage Format

SVD of A and Eigenvalues of A^*A

- Intuitive idea for computing the SVD of $A \in \mathbb{C}^{m\times n}$ (assuming $m \geq n$):
  - Form A^*A and compute its eigenvalue decomposition $A^*A = V\Lambda V^*$.
  - Let $\Sigma = \sqrt{\Lambda}$, i.e., $\mathrm{diag}(\sqrt{\lambda_1}, \sqrt{\lambda_2}, \dots, \sqrt{\lambda_n})$, with λ_i sorted in descending order.
  - For non-zero singular values ($\sigma_k > 0$), compute the corresponding columns of U via $u_k = \frac{1}{\sigma_k} A v_k$. For zero singular values, extend U to form an orthonormal basis.
- This method is efficient if $m \gg n$.
- However, it may not be stable, especially for smaller singular values, because of the squaring of the condition number.
  - For the SVD of A computed via stable algorithms, $|\tilde{\sigma}_k - \sigma_k| = \mathcal{O}\left(\epsilon_{\text{machine}} \|A\|\right)$, where $\tilde{\sigma}_k$ and σ_k denote the computed and exact kth singular values.
  - If computed from the eigenvalue decomposition of A^*A, $|\tilde{\sigma}_k - \sigma_k| = \mathcal{O}\left(\epsilon_{\text{machine}} \|A\|^2/\sigma_k\right)$, which is problematic if $\sigma_k \ll \|A\|$.
- If one is interested in only relatively large singular values, then computing the eigenvalues of A^*A is not a problem. For general situations, a more stable algorithm is desired.

A Different Reduction to Eigenvalue Problem

- Typical algorithms for computing the SVD are similar to those for computing eigenvalues.
- For simplicity, assume $A \in \mathbb{C}^{m \times m}$ is a square matrix. The Hermitian matrix $H = \begin{bmatrix} 0 & A^* \\ A & 0 \end{bmatrix}$ has the eigenvalue decomposition:

$$H \left(\frac{1}{\sqrt{2}} \begin{bmatrix} V & V \\ U & -U \end{bmatrix} \right) = \left(\frac{1}{\sqrt{2}} \begin{bmatrix} V & V \\ U & -U \end{bmatrix} \right) \begin{bmatrix} \Sigma & 0 \\ 0 & -\Sigma \end{bmatrix}$$

  where $A = U \Sigma V^*$ gives the SVD. This approach is stable.
- In practice, such a reduction is done implicitly without explicitly forming the large matrix H.
- Typically done in two phases.

Two-Phase Method

- In the first phase, reduce to bidiagonal form by applying different orthogonal transformations on the left and right, which requires $\mathcal{O}\left(mn^2\right)$ flops.
- In the second phase, reduce to diagonal form using a variant of the QR algorithm or the divide-and-conquer algorithm, which requires $\mathcal{O}\left(n^2\right)$ flops for fixed precision.

$$\underset{A}{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{bmatrix}} \xrightarrow{\text{Phase 1}} \underset{B}{\begin{bmatrix} \times & \times & & \\ & \times & \times & \\ & & \times & \times \\ & & & \times \\ & & & \\ & & & \end{bmatrix}} \xrightarrow{\text{Phase 2}} \underset{\Sigma}{\begin{bmatrix} \times & & & \\ & \times & & \\ & & \times & \\ & & & \times \\ & & & \\ & & & \end{bmatrix}}$$

- We hereafter focus on the first phase.

Golub–Kahan Bidiagonalization

- Apply Householder reflectors on both the left and right sides.

$$\underset{A}{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{bmatrix}} \xrightarrow{U_1^* \cdot} \underset{U_1^* A}{\begin{bmatrix} \mathbf{\times} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \\ \mathbf{0} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \\ \mathbf{0} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \\ \mathbf{0} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \\ \mathbf{0} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \\ \mathbf{0} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \end{bmatrix}} \xrightarrow{\cdot V_1} \underset{U_1^* A V_1}{\begin{bmatrix} \times & \mathbf{\times} & \mathbf{0} & \mathbf{0} \\ & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \\ & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \\ & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \\ & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \\ & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \end{bmatrix}} \xrightarrow{U_2^* \cdot} \underset{U_2^* U_1^* A V_1}{\begin{bmatrix} \times & \times & & \\ & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \\ & \mathbf{0} & \mathbf{\times} & \mathbf{\times} \\ & \mathbf{0} & \mathbf{\times} & \mathbf{\times} \\ & \mathbf{0} & \mathbf{\times} & \mathbf{\times} \\ & \mathbf{0} & \mathbf{\times} & \mathbf{\times} \end{bmatrix}} \xrightarrow{\cdot V_2} \underset{U_2^* U_1^* A V_1 V_2}{\begin{bmatrix} \times & \times & & \\ & \times & \mathbf{\times} & \mathbf{0} \\ & & \mathbf{\times} & \mathbf{\times} \\ & & \mathbf{\times} & \mathbf{\times} \\ & & \mathbf{\times} & \mathbf{\times} \\ & & \mathbf{\times} & \mathbf{\times} \end{bmatrix}}$$

- Work for Golub–Kahan bidiagonalization requires $\sim 4mn^2 - \frac{4}{3}n^3$ flops.

Lawson–Hanson–Chan Bidiagonalization

- Speed up by first performing a QR factorization on A.
- Work for LHC bidiagonalization requires $\sim 2mn^2 + 2n^3$ flops, which is advantageous if $m \geq \frac{5}{3}n$.

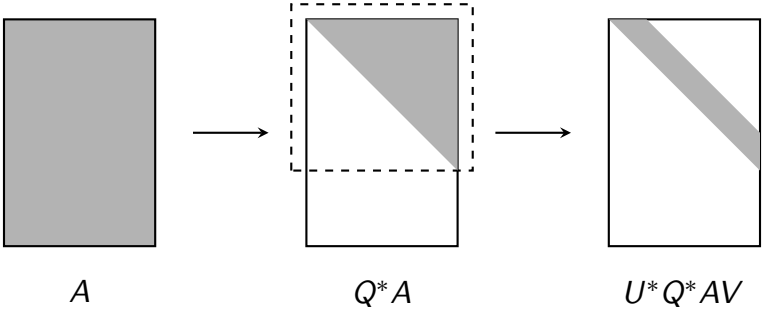
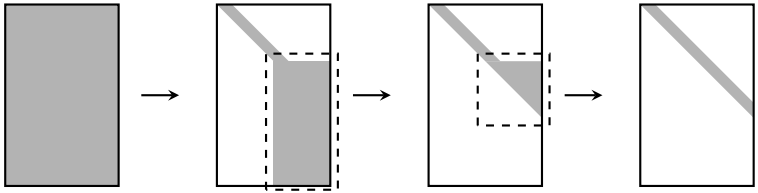


Figure: Lawson–Hanson–Chan Bidiagonalization

Three-Step Bidiagonalization

- Hybrid approach: Apply a QR factorization at a suitable time on a submatrix with a $2:1$ aspect ratio.
- Work for three-step bidiagonalization requires $\sim 4mn^2 - \frac{4}{3}n^3 - \frac{2}{3}(m-n)^3$ flops.



A $\quad$ $U_1^* A V_1$ $\quad$ $Q^* U_1^* A V_1$ $\quad$ $U_2^* Q^* U_1^* A V_1 V_2$

Figure: Three-Step Bidiagonalization

Comparison of Performance

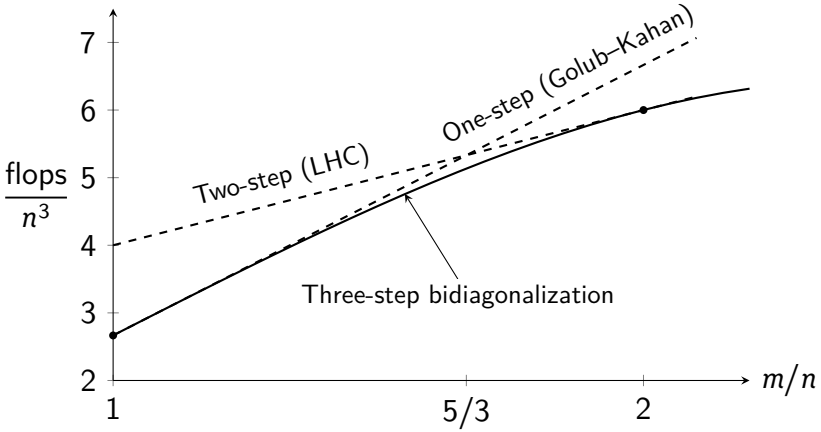


Figure: Comparison of Performance

Outline

1. Computing the SVD (NLA§31)
2. **Sparse Storage Format**

Sparse Linear System

- Boundary value problems and implicit methods for time-dependent PDEs yield large systems of linear algebraic equations to solve.
- A matrix is *sparse* if it has relatively few nonzero entries.
- Sparsity can be exploited to use far less than the $\mathcal{O}\left(n^2\right)$ storage and $\mathcal{O}\left(n^3\right)$ work required in the standard approach for solving a system with a dense $n \times n$ matrix.

Storage Format of Sparse Matrices

- Sparse matrices are typically stored in special formats that store only the nonzero entries, along with indices to identify their locations in the matrix, such as:
  - compressed-row storage (CRS) (aka compressed sparse row (CSR))
  - compressed-column storage (CCS) (aka compressed sparse column (CSC))
  - block compressed row storage (BCRS)
- Banded matrices have their own special storage formats (such as Compressed Diagonal Storage (CDS)).
- See survey at http://netlib.org/linalg/html_templates/node90.html.
- Explicitly storing indices incurs additional storage overhead and makes arithmetic operations on nonzeros less efficient due to indirect addressing. Thus, these formats are beneficial only for very sparse matrices.
- The choice of storage format can have a big impact on the effectiveness of different versions of the same algorithm (e.g., with different loop orderings).
- Besides direct methods, these storage formats are also important in implementing iterative and multigrid solvers.

Example of Compressed Sparse Row (CSR) Storage

CSR Representation Example

Matrix A:

$$A = \begin{pmatrix} 10 & 0 & 0 & 0 & -2 & 0 \\ 3 & 9 & 0 & 0 & 0 & 3 \\ 0 & 7 & 8 & 7 & 0 & 0 \\ 3 & 0 & 8 & 7 & 5 & 0 \\ 0 & 8 & 0 & 9 & 9 & 13 \\ 0 & 4 & 0 & 0 & 2 & -1 \end{pmatrix}$$

CSR Representation (1-Based Indexing):

- **val:** `[10, -2, 3, 9, 3, 7, 8, 7, 3, 8, 7, 5, 8, 9, 9, 13, 4, 2, -1]`
- **col_ind:** `[1, 5, 1, 2, 6, 2, 3, 4, 1, 3, 4, 5, 2, 4, 5, 6, 2, 5, 6]`
- **row_ptr:** `[1, 3, 6, 9, 13, 17, 20]`